

Classes e Objetos

1 - Instalação da Linguagem de Programação e do Ambiente de Desenvolvimento

Duas características principais são responsáveis pela enorme popularidade do paradigma orientado a objetos: (a) o mapeamento de entidades do mundo real em unidades de programação, que são utilizadas como novos tipos da linguagem; e (b) a capacidade de definir unidades de programação com dados e métodos (funções que acessam esses dados) comuns em unidades mais genéricas e herdá-los em outras unidades mais especializadas.

Python é uma linguagem orientada a objetos cuja utilização tem crescido muito atualmente. Suas principais características são: independência de plataforma, gerenciamento automático da utilização de memória e transparência no endereçamento de memória. Python provê uma plataforma de desenvolvimento com uma extensa API (Application Program Interface), que suporta diversas funcionalidades. Várias outras linguagens suportam o paradigma orientado a objetos, como por exemplo: Java, C++, PHP, JavaScript, Python, Pearl, Delphi, C#, Visual Basic. Dominar o paradigma orientado a objetos é muito útil no aprendizado de outras linguagens.

A manipulação de conjuntos de dados em Python é muito poderosa, favorecendo a criação de um código compacto e versátil, o que tem contribuído para grande popularidade atual desta linguagem. Além disso, esta linguagem está sendo largamente utilizada para o desenvolvimento de software científico, como por exemplo, aprendizado de máquinas (*machine learning*) e aprendizado profundo (*deep learning*) que estão potencializando aplicações muito bem sucedidas na área de inteligência artificial.

Para fixar os conceitos que serão apresentados neste tutorial, é fundamental que cada trecho de código apresentado seja implementado e testado. Desta forma, será necessário instalar:

- a versão estável mais recente do Python;
- e a versão mais recente do IDE (*Integrated Development Environment* - Ambiente de Desenvolvimento Integrado) PyCharm da Jet Brains.

No site <https://www.python.org/>, selecione a opção [Downloads](#), baixe e instale no seu computador o arquivo executável da versão mais recente do Python para Windows (por exemplo: [python-3.11.5-amd64](#)).

No site <https://www.jetbrains.com/pt-br/pycharm/> selecione a opção [Baixar](#) e novamente esta opção para a baixar a versão gratuita Community do PyCharm, baixe e instale em seu computador o arquivo executável da versão mais recente do PyCharm Community (por exemplo: [pycharm-community-2023.2.1](#)).

Para implementar e testar os trechos de código das seções 3 e 4 deste tutorial, que ilustram conceitos básicos da linguagem Python, crie um projeto no IDE PyCharm com o nome **MP-E1**:

- selecione **File -- New Project**
- na janela **Create Project**
 - mantenha a seleção de **New environment using** para criar um ambiente específico para o seu projeto
 - em **Location**
 - navegue para escolher um diretório para armazenar o seu projeto
 - complete o diretório com o nome do projeto : **MP-E1**
 - em **Basic Interpreter**
 - selecione a versão Python **3.11**
 - mantenha a seleção de **Create a main.py welcome script**
 - finalize a criação ativando o botão **Create**

Crie um diretório padrão para armazenar os diretórios e arquivos fontes do seu projeto:

- selecione a aba **Project** no canto esquerdo da tela para visualizar a janela lateral **Project**
- clique na linha com o nome do seu projeto e selecione, com o botão direito do mouse
 - **New -- Directory**
 - preencha com o nome **src** (abreviação para source)
 - diretório onde tradicionalmente são armazenados os arquivos fontes do projeto
 - em geral organizados em subdiretórios de **src**
- assinale o diretório **src** como raiz dos arquivos fontes do projeto
 - selecione o diretório **src** com o botão direito do mouse
 - escolha a opção: **Mark Directory as -- Mark as Source Root**
- selecione o arquivo **main.py** e mova-o para o diretório **src**

Ajuste o arquivo **main.py**:

- na janela de projeto (lateral esquerda)
 - clique no arquivo **main.py** para visualizar o seu código na janela à direita da janela de projeto
- remova a seleção de ponto de parada para teste passo a passo (debug)
 - clicando no círculo vermelho no lado esquerdo de uma das linhas do código
- remova todos os comentários : strings iniciados com o caracter **#**
- altere o código do arquivo **main.py** para ficar da seguinte forma:

```
def imprimir(linguagem_programação):
    print(f'Vamos aprender a programar na linguagem {linguagem_programação}.')

if __name__ == '__main__':
    imprimir('Python')
```

Execute o programa, clicando na seta verde no canto superior direito da sua janela:



Você visualizará o resultado do processo na janela Run (execução)

```
Vamos aprender a programar na linguagem Python.
Process finished with exit code 0
```

O código em Python é organizado em blocos de comandos. Diferentemente de outras linguagens (como: Java, C++ e C), bloco em Python não são delimitados por `{ }`, e sim por indentação (utilizando `Tab`). Portanto, você deve estar muito atento para que a indentação de seu código esteja correta.

Na execução, o Python atribui à variável pré-definida `__name__` o string `'__main__'`, e executa o corpo principal do programa, que neste caso tem somente o comando condicional `if`. Variáveis pré-definidas são cercadas por duplo underline, como é o caso da variável `__name__`.

O comando condicional `if` testa com sucesso que a variável `__name__` é igual ao (contém o mesmo valor que) string `'__main__'`. Pelo fato do teste do comando `if` ser bem sucedido, o seu corpo interno de comandos é executado: neste exemplo, somente a chamada da função imprimir, passando como argumento o string `'Python'`.

A função imprimir recebe o parâmetro (entre parênteses) `linguagem_programação`, para o qual foi atribuído como argumento o string `'Python'`. Ao ser chamada a função executa o seu corpo interno de comandos, que neste exemplo, é composto somente pela chamada da função pré-definida (*builtin*) `print`, utilizada para imprimir strings. Funções pré-definidas são disponíveis em Python sem que seja necessário defini-las.

O argumento passado para a função `print` é um string formatado (`f'...'`) para compor trechos fixos (`'Vamos aprender a programar na linguagem '` e `'.'`) com trechos variáveis (entre chaves: `{ }`). O trecho variável recebe o valor passado como argumento (`'Python'`) para o parâmetro `linguagem_programação`. Como resultado é impresso na tela do seu computador, o string: `Vamos aprender a programar na linguagem Python`.

O resultado da execução é mostrado na janela de execução (`Run`). Adicionalmente, como mensagem de final de execução é impressa a mensagem: `Process finished with exit code 0`, cuja tradução é: `Processo concluído com código de saída 0`. O código `0` indica que a execução foi concluída com sucesso. Caso seja reportado o código `1`, a mensagem indica execução mal sucedida. Esta mensagem se repete em todas as execuções e será omitida nas ilustrações da seção 2.

Este exemplo muito simples, com uma breve explicação, é o seu primeiro contato com a execução de um programa em Python. Na próxima seção, vamos conceituar a primeira etapa de um projeto de referência.

2 - Implementação de Entidades Sem Relacionamentos

Nos tutoriais dessa disciplina é adotado o modelo de aprendizado a partir de um projeto de referência, cujo código se inicia neste tutorial e cresce incrementalmente até atingir o projeto completo no Tutorial 4. Desta forma, a especificação do projeto, definida no Tutorial Auxiliar A - Especificação de um Projeto Orientado a Objetos de Referência, será implementada ao longo dos quatro tutoriais da disciplina.

Neste ponto é importante observar uma diferença fundamental entre Python e outras linguagens de programação como C, C++ e Java. Python é uma linguagem com tipagem dinâmica, o que significa que você não especifica o tipo de dado sendo definido. Esse tipo será inferido durante a execução do código. Vamos considerar, por exemplo, que os valores atribuídos a uma variável podem ser dos seguintes tipos:

- **str** : string --- ex: `disciplina = 'Metologia de Programação'`;
- **int** : inteiro positivo ou negativo --- ex: `carga_horária = 72`;
- **float** : ponto flutuante positivo ou negativo --- ex: `nota_mínima_aprovação = 6.0`;
- **bool** : boolean com valores True ou False --- ex: `disciplina_obrigatória = True`.

Cada entidade do mundo real é mapeada em uma classe de objetos (denominada simplesmente como classe) em uma linguagem de programação orientada a objetos. Uma classe é composta de dados que mapeiam atributos do mundo real. Uma classe é um tipo complexo (composto de vários dados) que estende os tipos comuns da linguagem de programação (inteiro: **int**, flutuante: **float**, booleano: **bool**, string: **str**), acrescentando tipos mapeados a partir das entidades do mundo real relevantes para um dado projeto de software. Assim como podemos criar uma variável inteira, flutuante (números com casas decimais) booleana ou string, podemos criar variáveis a partir de uma classe. Essas variáveis são denominadas objetos e, em geral, são inicializadas com valores para os atributos da classe.

Vamos adotar a seguinte metodologia para você entender cada trecho de código que será apresentado: (a) primeiro ilustramos o trecho de código, e (b) em seguida, vamos explicar cada novo conceito que ele apresenta. Mesmo que você já conheça alguns desses conceitos, não pule nada. Leia tudo com atenção e releia se for necessário até ficar claro para você. Alguns conceitos serão comentados mais de uma vez, mas isso é bom para fixar bem o entendimento a respeito.

No Tutorial Auxiliar [Especificação de um Projeto Orientado a Objetos de Referência](#), foi especificado o projeto [Divulgação de Lançamentos de Veículos](#), que servirá de referência para a especificação dos projetos individuais dos alunos, e terá seu código desenvolvido incrementalmente nos 4 tutoriais dessa disciplina. A leitura do Tutorial Auxiliar é um pré-requisito para a leitura do Tutorial 1.

Os relacionamentos entre as entidades do projeto [Divulgação de Lançamentos de Veículos](#) são os seguintes:

- Entidades sem relacionamentos : Veículo - AgênciaPublicidade
- Relacionamento Múltiplo : Montadora [1:n] Veículo
- Associação : Lançamento -- Montadora - Veículo - AgênciaPublicidade
- Herança : Veículo -- Carro - Caminhão
- Delegação : Lançamento -- Data

Neste tutorial, vamos ilustrar a implementação das entidades sem relacionamentos: [Veículo](#) e [AgênciaPublicidade](#).

No arquivo `veiculo` do diretório `entidades`, vamos definir a classe `Veículo`. Observe que a classe `Veículo` mapeia em uma unidade computacional (classe), uma entidade que você encontra no mundo real, e que seus atributos (`marca_modelo`, `alimentação`, `potência`, `importado`) também mapeiam características de um veículo, que você encontra no mundo real.

`class Veículo:`

```
def __init__(self, marca_modelo, alimentação, potência, importado):
    self.marca_modelo = marca_modelo
    self.alimentação = alimentação if alimentação in ('flex', 'diesel', 'elétrico') else 'indefinida'
    self.potência = potência
    self.importado = importado
```

A palavra reservada `class` (classe) é utilizada para definir uma classe cujo nome deverá utilizar a primeira letra maiúscula (ex: `Carro`) ou um nome composto de palavras com a primeira letra maiúscula (ex: `OficinaMecânica`). Nomes de diretórios, arquivos, funções e variáveis, utilizam somente letras minúsculas (ex: `nome`), também separando palavras por underline (ex: `data_nascimento`).

A palavra reservada `def` (abreviação para `define function`) é utilizada para definir o método `__init__`. Funções podem ser utilizadas para obter, armazenar, ou computar dados a partir de outros dados. Funções internas a uma classe são chamadas de métodos, no jargão orientado a objetos. Métodos padronizados em Python tem seu nome cercado por dois underlines, como por exemplo o método `__init__`. Os atributos da classe são inicializados neste método, que é chamado quando o objeto é construído. Por esse motivo os métodos `__init__`, utilizados para construir objetos nas classes, são chamados de construtores. Esse método tem, em geral, um parâmetro para cada atributo do objeto que será inicializado na sua construção.

Todos os métodos de uma classe tem como primeiro parâmetro a palavra reservada `self` (próprio) para indicar que todo método da classe tem acesso a todos os dados da classe, que são prefixados por `self` (ex: `self.marca_modelo`). Os demais parâmetros do método `__init__` são utilizados para inicializar os atributos da classe (ex: `self.marca_modelo = marca_modelo`). O caracter `=` é utilizado para atribuir valores ou expressões a uma variável.

O atributo `self.alimentação` é considerado um enumerado, ou seja, uma variável que pode assumir um valor definido em uma lista de valores. Neste caso, é importante verificar, utilizando um comando `if` (se) associado ao comando de atribuição, se o valor passado como parâmetro pertence à lista de valores possíveis, correspondendo aos tipos de alimentação utilizadas em um veículo.

```
self.alimentação = alimentação if alimentação in ('flex', 'diesel', 'elétrico') else 'indefinida'
```

O comando `if` é um teste condicional, utilizado para testar um condição. Por exemplo se queremos testar se um string não é nulo (`None`) antes de imprimi-lo. É necessário utilizar o separador `:` após a condição do `if` (se) e após o `else` (senão). Se a condição for verdadeira o bloco de comandos que segue logo após o separador `:`, localizado após condição, será executado. Caso contrário, será executado o bloco de comando que segue logo após o `else`. Ao comparar com `None`, não utilizamos `==` ou `!=` (utilizados para comparar com variáveis ou valores do tipo `int`, `float`, `bool` ou `str`) e sim: `is` ou `is not`.

```
if marca_modelo is not None:
    print(marca_modelo)
else:
    print('marca modelo com valor nulo')
```

No exemplo anterior, o bloco de comandos associado à condição verdadeira do `if` ou à condição falsa (`else`) tem um único comando. Neste caso particular, utilizaremos neste tutorial os comandos na mesma linha do `if` e do `else`, mas você pode optar pela formatação que deseja utilizar.

```
if marca_modelo is not None: print(marca_modelo)
else: print(' marca modelo com valor nulo')
```

É muito útil que um objeto seja identificado unicamente pelo valor do seu atributo chave. Para classes representando pessoas (ex: Cliente, Funcionário, Aluno, Médico) o atributo chave mais utilizado é o CPF, uma vez que identifica uma pessoa unicamente em todo o território nacional. No entanto, dependendo da aplicação podemos utilizar o nome da pessoa, desde que não exista outra pessoa com o mesmo nome no projeto.

No caso, do projeto [Divulgação de Lançamentos de Veículos](#), o lançamento de um veículo com uma determinada `marca modelo` será divulgado uma única vez, quando for lançado pela primeira vez no mercado brasileiro. Desta forma, a `marca modelo` do veículo lançado servirá como chave para diferenciá-lo de todos os outros lançamentos.

Estamos assumindo que a montadora, do veículo que será lançado, contratará uma agência de publicidade para divulgar o seu lançamento. No arquivo `agência_publicidade` do diretório `entidades`, vamos definir a entidade `AgênciaPublicidade` do mundo real, em sua unidade computacional: a classe `AgênciaPublicidade`.

```
class AgênciaPublicidade:
    def __init__(self, nome, país_origem, ranking_avaliação):
        self.nome = nome
        self.país_origem = país_origem
        self.ranking_avaliação = ranking_avaliação
```

É praticamente uma convenção entre as linguagens orientadas a objetos, a definição de classes auxiliares que forem úteis para aplicações em geral em um diretório denominado `util`. Portanto, definiremos a classe `Data` no módulo `data` do diretório `util` (embora seja acentuado em português, utilizaremos o nome em inglês, consagrado nas linguagens de programação). A seguir, a implementação do módulo `data`, com a classe auxiliar `Data` e uma função associada a ela.

```
from datetime import date
```

```
class Data:
    def __init__(self, dia, mês, ano):
        self.dia = dia
        self.mês = mês
        self.ano = ano

    def __str__(self):
        if self.dia < 10: data_str = '0' + str(self.dia)
        else: data_str = str(self.dia)
        if self.mês < 10: data_str += "/0" + str(self.mês) + "/"
        else: data_str += "/" + str(self.mês) + "/";
        data_str += str(self.ano)
        return data_str

    def __eq__(self, data):
        if self.dia == data.dia and self.mês == data.mês and self.ano == data.ano: return True
        return False

    def __ne__(self, data): return not self == data
```

```
def __gt__(self, data):
    if self.ano > data.ano: return True
    elif self.ano < data.ano: return False
    if self.mês > data.mês: return True
    elif self.mês < data.mês: return False
    if self.dia > data.dia: return True
    elif self.dia < data.dia: return False
    return False
```

```
def __lt__(self, data):
    if self.ano < data.ano: return True
    elif self.ano > data.ano: return False
    if self.mês < data.mês: return True
    elif self.mês > data.mês: return False
    if self.dia < data.dia: return True
    elif self.dia > data.dia: return False
    return False
```

```
def __ge__(self, data):
    if self < data: return False
    else: return True
```

```
def __le__(self, data):
    if self > data: return False
    else: return True
```

```
def calcular_idade(self, data_referência=None):
    if data_referência is None:
        dia_atual_str, mês_atual_str, ano_atual_str = date.today().strftime("%d/%m/%Y").split('/')
        dia_referência, mês_referência, ano_referência = int(dia_atual_str), int(mês_atual_str), int(ano_atual_str)
    else:
        dia_referência, mês_referência, ano_referência = data_referência.dia, data_referência.mês, data_referência.ano
    idade = ano_referência - self.ano
    if mês_referência < self.mês or (mês_referência == self.mês and dia_referência < self.dia): idade -= 1
    return idade
```

Na implementação do método `__str__` está sendo checado se o dia ou mês tem apenas um dígito decimal, para gerar um string iniciando com o caracter '0'. Os strings gerados para representar dia, mês e ano estão sendo separados pelo caracter '/' para gerar o string completo da data.

Para que objetos de uma classe possam ser comparados, o que é muito utilizado em filtragem de objetos, você pode implementar métodos padronizados de comparação. Na classe `Data`, esta possibilidade é muito útil, para que você possa comparar se dois objetos da classe `Data` tem a mesma data, ou se a data de uma deles é superior ou inferior à data do outro.

Para podermos verificar se dois objetos são iguais utilizando o comparador de igualdade `==`, precisamos implementar o método padronizado `__eq__`. Desta forma, a comparação entre duas datas, `if data1 == data2` é executada como se fosse `if data1.__eq__(data2)`. Data equivalentes devem ter os mesmos dias, meses e anos.

```
def __eq__(self, data):
    if self.dia == data.dia and self.mês == data.mês and self.ano == data.ano: return True
    return False
```

Essa implementação não funciona para a comparação: `if data == None`. Utilizar o comparador `==` na implementação do método `__eq__` para comparar data com `None` não pode ser feita dentro do próprio método `__eq__` (equal) porque geraria um loop infinito de chamadas recursivas do próprio método `__eq__`, uma vez que este método é chamado toda vez que se utiliza o comparador `==` para comparar um objeto da classe `Data`. Para evitar essa situação, a forma correta de comparar uma objeto `Data` com `None` é: `if data is not None`.

Para podermos verificar se dois objetos são diferentes utilizando o comparador de desigualdade `!=`, precisamos implementar o método padronizado `__ne__` (not equal). Agora já podemos utilizar o comparador `==` na implementação de `__ne__` por que sua chamada vai gerar a chamada do método `__eq__` e, portanto, não haverá chamada recursiva. Desta forma, sua implementação fica muito simplificada.

Para completar precisamos implementar os seguintes métodos padronizados de comparação:

- `__gt__` (greater than : maior que) para `>` ;
- `__ge__` (greater or equal: maior ou igual) para `>=` ;
- `__lt__` (lesser than : menor que) para `<` ;
- `__le__` (lesser or equal : menor ou igual) para `<=` .

Obviamente não faz sentido comparar se uma data é maior ou menor que `None` e, portanto, podemos utilizar a comparação de dias, meses e anos para implementar esses quatro métodos padronizados de comparação, simplificando as implementações. Para os métodos `__gt__` e `__lt__`, inicialmente comparamos os anos, se forem iguais, comparamos os meses e se forem iguais comparamos os dias. As implementações dos métodos `__ge__` e `__le__` ficam bem simples, porque se utilizam das implementações de `__gt__` e `__lt__`.

É importante implementar o método `calcular_idade` a partir da data de nascimento de uma pessoa. Inicialmente obtemos os strings correspondentes ao dia, mês e ano da data atual e convertemos esses strings para valores inteiros. Inicialmente subtraímos o ano atual do ano da data de nascimento. Se a pessoa ainda não fez aniversário neste ano, basta subtrairmos 1 deste valor. Caso contrário, este valor já corresponderá à idade da pessoa.

Outra entidade genérica muito utilizada em vários projetos, é entidade `Endereço`. Essa entidade pode ser utilizada para pessoas residentes no mesmo país tem os seguintes atributos: `logradouro` (rua, avenida, praça, etc), `número`, `complemento` (opcional -- ex: apto 704, fundos), `bairro`, `cidade`, `uf`, e `cep`.

3 - Método para gerar um String representando os Atributos do Objeto

Após cadastrar vários objetos de uma dada classe, é muito comum desejarmos imprimir os dados de cada objeto cadastrado. Para obter o string que representa os atributos do objeto criado, utilizamos o método padronizado `__str__`. A seguir, vamos acrescentar o método `__str__` na classe `Veículo` e na classe `AgênciaPublicidade`, omitindo (somente para economizar espaço no Tutorial) o método `__init__`, que é sempre o primeiro método a ser implementado na classe.

`class Veículo:`

```
def __str__(self):
    if self.importado: importado_str = 'importado |'
    else: importado_str = ''
    formato = '{:<19} {} {:<8} {} {:<6} {} {}'
    veículo_formatado = formato.format('|', self.marca_modelo, '|', self.alimentação, '|', f'{self.potência:3d}' + ' cv', '|',
                                     importado_str)
    return veículo_formatado
```


O método `__str__` não tem parâmetros, dado que sua função é gerar um string a partir dos atributos da classe. Em geral, os valores dos atributos da classe que serão impressos, serão representados por strings de tamanhos diferentes (ex: marca modelo de veículos). Para tornar a impressão mais amigável é interessante alinhar cada coluna de valores a ser impressa. No caso de veículos as colunas são: `marca_modelo`, `alimentação`, `potência` e `importado`. Python suporta a definição do formato que será utilizado para imprimir cada objeto. Esse formato é definido por um string contendo para cada coluna a ser impressa, calculado como o tamanho do maior valor a ser impresso em cada coluna, acrescido de alguns espaços para separar as colunas.

No primeiro comando do método `__str__`, vemos a sintaxe utilizada para definir o formato de impressão, assinalando o espaço de cada uma das quatro colunas a serem impressas para cada objeto da classe `Veículo`. O caracter `<` é utilizado para indicar indentação à esquerda, ou seja todos os valores de uma dada coluna serão alinhados no início de cada string da coluna. Se a escolha tivesse sido indentação à direita, seria utilizado o caracter `>`, e neste caso todos os valores de uma dada coluna seriam alinhadas ao final de cada string da coluna.

Após a definição do string do formato, o método `format` (suportado pelo Python) é aplicado ao formato, recebendo como parâmetros os strings de cada coluna a ser impressa. Desta forma, os atributos que não são strings (ex: idade e peso) foram convertidos para string pela utilização da função `str`, pré-definida pelo Python.

Os valores da coluna `potência` dos veículos que serão cadastrados poderão ocupar 2 ou 3 dígitos. Para alinhar esses valores à esquerda, é possível utilizar o formatador `f{}`, informando quantos caracteres deverão ser impressos (neste caso 3 caracteres: `3d`).

Para separar as colunas da impressão dos objetos é utilizado o caracter `|`. Para esse caracter não é necessário definir a quantidade de caracteres, utilizando somente `{}`.

```
Veículos : marca modelo - alimentação - potência - importado
1 - | Hyundai HB20          | flex      | 80 cv |
2 - | Chevrolet Onix Plus  | flex      | 116 cv |
```

Quando uma classe tiver um atributo do tipo `bool`, o mais recomendado é formatar o seu valor na última coluna e mostrá-lo somente quando seu valor for `True`. Observe que, no método `__str__` foi testado se o valor do atributo `importado` é `True`, e neste caso é incluída uma coluna com a palavra `importado`. Caso contrário, o valor é omitido; o que mais amigável do que mostrar `não importado`, para a maioria dos veículos que não são importados.

```
Veículos : marca modelo - alimentação - potência - importado
1 - | Hyundai HB20          | flex      | 80 cv |
2 - | Chevrolet Onix Plus  | flex      | 116 cv |
3 - | Fiat Strada          | diesel    | 130 cv |
4 - | Fiat Toro            | diesel    | 185 cv |
5 - | BYD Dolphin          | elétrico  | 95 cv | importado |
```

A seguir, vamos ilustrar o método `__str__` da classe `AgênciaPublicidade`.

```
class AgênciaPublicidade:
    def __str__(self):
        formato = '{<10} {<14} {<1}'
        agência_publicidade_formatada = formato.format('|', self.nome, '|', self.país_origem, '|', str(self.ranking_avaliação), '|')
        return agência_publicidade_formatada
```

4 - Conjuntos para Armazenar Objetos da Classe

A forma mais simples de armazenar objetos (variáveis) criadas a partir de uma classe (novo tipo da aplicação que mapeia uma entidade do mundo real) é definir uma variável global (pode ser utilizada por toda a aplicação) utilizando o nome da classe no plural (ex: `veículos`, `agências_publicidade`), dado que irá armazenar um conjunto de objetos. Python suporta uma estrutura de dados denominada lista. Os itens (ex: objetos da classe `Veículo`) armazenados na lista (ex: `veículos`) podem ser consultados por índices na ordem em que foram armazenados (ex: `veículos[n]`, onde varia de 0 até o número de itens da lista menos 1).

No arquivo `veículo`, definimos antes da definição da classe `Veículo`, o conjunto de objetos `veículos` e suas funções associadas.

```
veículos = []

def get_veículos(): return veículos

def inserir_veículo(veículo): veículos.append(veículo)
```

O conjunto `veículos` é inicializado com uma lista vazia (`[]`). A função `get_veículos` retorna o conjunto de veículos. A função `inserir_veículo` recebe um objeto `veículo` e apenda na lista `veículos`. Pelo fato de cada função ter um único comando, optamos por defini-lo na mesma linha da definição da função. No entanto, se você preferir pode indentá-lo na próxima linha da função:

```
def get_veículos():
    return veículos
```

A seguir, implementamos no arquivo `agência_publicidade`, antes da classe `AgênciaPublicidade` o conjunto de objetos `agências_publicidade` e suas funções associadas.

```
agências_publicidade = []

def get_agências_publicidade(): return agências_publicidade

def inserir_agência_publicidade(agência_publicidade): agências_publicidade.append(agência_publicidade)
```

5 - Utilizando Filtros para Selecionar um Subconjunto de Objetos

Para que um projeto seja útil para os usuários, ele deve suportar a filtragem dos objetos cadastrados.

No arquivo `veículo` vamos acrescentar a função `selecionar_veículos`.

```
def selecionar_veículos(importado=None, alimentação=None, potência_máxima=None):
    filtros = '\nFiltros -- '
    if importado: filtros += 'importado'
    elif importado == False: filtros += 'nacional'
    if alimentação is not None: filtros += ' - alimentação: ' + alimentação
    if potência_máxima is not None: filtros += ' - potência máxima: ' + str(potência_máxima)
    veículos_selecionados = []
    for veículo in veículos:
        if importado in (True, False) and veículo.importado != importado: continue
        if alimentação is not None and veículo.alimentação != alimentação: continue
        if potência_máxima is not None and veículo.potência > potência_máxima: continue
        veículos_selecionados.append(veículo)
    return filtros, veículos_selecionados
```

Antes de detalhar a função de filtragem, vamos comentar o uso dos comandos `if` e `for` que são utilizados nessa função.

Vimos anteriormente que o teste condicional `if` pode ser utilizado com a palavra reservada `else` para indicar o bloco de ações que será realizado se a condição que está sendo testada pelo `if` não for satisfeita. No entanto, na função de filtragem, caso a condição do `if` não seja aceita, será necessário testar uma outra condição, dado que o valor do parâmetro importado poderá ser `None`, `True` ou `False`. Neste caso, é utilizada a palavra reservada `elif`, que é uma contração para as palavras `else if` (senão se), associada a uma segunda condição. Na situação mais geral, um comando condicional pode testar várias condições, utilizando um `if` (para testar a primeira condição), vários `elif` (para testar as demais condições) e um `else` (se nenhuma das condições for aceita).

O comando `for` é utilizado para testar uma condição que resultará na repetição da execução de um bloco de comandos, enquanto a condição testada permanecer verdadeira. O comando `for`, utilizado conjuntamente com a palavra reservada `in`, indica que a cada passada do loop o objeto (`veículo`) será atualizado com o valor do próximo elemento do conjunto (`veículos`). No primeiro comando interno ao loop, é verificado (pelo comando `if`) se o filtro `importado` corresponde a um dos elementos da tupla constituída por `True` e `False`. Uma tupla é uma lista constante, que após definida não pode ter nenhum elemento inserido ou removido.

```
for veículo in veículos:
    if importado in (True, False) and veículo.importado is not importado: continue
```

Bem, agora vamos detalhar a função `selecionar_veículos`. Os parâmetros dessa função são os filtros que serão utilizados para seleção. Os filtros são opcionais. Caso o valor do filtro não seja passado como argumento, já está previamente atribuído a `None`, para indicar que o implementador não deseja utilizar esse filtro na seleção.

Neste ponto, podemos observar a versatilidade da tipagem dinâmica de Python: o tipo do filtro `potência_máxima` poderá ser inicializado com valor `None` e posteriormente ser atribuído a um valor inteiro, sem nenhum problema. No entanto, essa facilidade só será utilizada para inicialização prévia com `None` para padronizar filtros não obrigatórios, mas você não deverá alterar os tipos dos valores atribuídos a uma mesma variável para não tornar o seu código mais difícil de ser entendido e testado.

Na parte inicial da função é construído um string para concatenar os valores dos filtros que forem passados como argumentos. Na parte final da função, é definida a variável `veículos_selecionados`, inicializada com uma lista vazia. A seguir, é definido o loop de seleção dos veículos cadastrados que serão testados para verificar se atendem os filtros passados como argumento. O objeto `veículo` que atender a todos filtros será inserido na lista `veículos_selecionados`. Ao final, a função irá retornar o string com os filtros solicitados (com valores diferentes de `None`) e a lista de veículos selecionados.

Agora, vamos entender como funciona o loop de seleção dos `veículos`. Para cada `veículo` da lista de alunos cadastrados, são checados os filtros, cujos valores foram passados pelo implementador. Para filtros com valor do tipo str, int, e float, os filtros são checados como obrigatórios verificando se são diferentes de `None`. Filtros do tipo `bool`, são checados como obrigatórios verificando se o valor passado pertence à tupla (`True, False`). Caso o filtro seja obrigatório, é checado se o filtro não é atendido e neste caso é utilizado o comando `continue` que faz com que a execução retorne ao início do loop para testar o próximo objeto `veículo`: o teste do `veículo` é interrompido (por que basta não atender um dos filtros, para ser descartado como selecionado), sem incluir o `veículo` testado na lista de selecionados.

Para testar o atendimento a um dado filtro, o atributo do aluno é verificado como diferente do valor do filtro, para executar o comando `continue` e interromper o teste do `veículo`, caso o atributo não corresponda ao valor do filtro. No caso de um filtro com valor do tipo `int` ou `float`, para o qual seja definido um valor de filtro como mínimo ou máximo será feita uma comparação para verificar que o filtro não atende esse valor mínimo (o atributo do objeto não pode ser inferior ao valor mínimo do filtro) ou máximo (ou o atributo do objeto não pode ser superior ao valor máximo do filtro). A estratégia de utilizar o comando `continue` para testar se o filtro não é atendido, simplifica a legibilidade do código gerado, definindo um único teste para cada filtro, sem a necessidade de utilizar encadeamentos de operadores booleanos (`and` e `or`) que tornariam a expressão mais complexa. Observe como ficaria o loop de seleção se não utilizássemos o comando `continue`:

```
for veículo in veículos:
    if (importado not in (True, False) or veículo.importado == importado)
    and (alimentação is None or veículo.alimentação == alimentação)
    and (potência_máxima is None or veículo.potência <= potência_máxima):
        veículos_selecionados.append(veículo)
```

Nesta alternativa de implementação, seria verificado para todos os filtros, se cada filtro não é obrigatório ou se a condição do filtro é atendida. Se todos os filtros forem atendidos, o objeto `veículo` é inserido na lista de selecionados.

Um boa estratégia utilizada para testar o funcionamento correto da seleção de alunos é começar não utilizando nenhum filtro como obrigatório (o que levará à seleção de toda a lista de alunos cadastrados) e ir acrescentando um filtro obrigatório de cada vez, mantendo os valores dos filtros utilizados anteriormente, de forma que a lista de alunos cadastrados diminua a medida que cada filtro adicional é tornado obrigatório. Vamos utilizar essa estratégia nos quatro tutoriais desta disciplina.

De forma semelhante, no arquivo `agenda_publicidade` vamos acrescentar a função `selecionar_agendas_publicidade`.

```
def selecionar_agências_publicidade(ranking_máximo_avaliação=None, país_origem=None, prefixo_nome=None):
    filtros = '\nFiltros -- '
    if ranking_máximo_avaliação is not None: filtros += 'ranking máximo de avaliação: ' + str(ranking_máximo_avaliação)
    if país_origem is not None: filtros += ' - país de origem: ' + str(país_origem)
    if prefixo_nome is not None: filtros += ' - prefixo do nome: ' + prefixo_nome
    agências_publicidade_selecionadas = []
    for agência_publicidade in agências_publicidade:
        if ranking_máximo_avaliação is not None and agência_publicidade.ranking_avaliação > ranking_máximo_avaliação: continue
        if país_origem is not None and agência_publicidade.país_origem != país_origem: continue
        if prefixo_nome is not None and not agência_publicidade.nome.startswith(prefixo_nome): continue
        agências_publicidade_selecionadas.append(agência_publicidade)
    return filtros, agências_publicidade_selecionadas
```

A função `selecionar_agendas_publicidade` não apresenta nenhum conceito novo em relação ao que já descrevemos na função `selecionar_veículos`.

6 - Definindo o Controle do Projeto

No arquivo `projeto`, do diretório `controle`, definimos as funções que controlam a execução do projeto, responsáveis pelo cadastro e pela filtragem dos objetos das classes `Veículo` e `AgênciaPublicidade`. Observe que o nome desse arquivo é baseado no nome do projeto que está sendo implementado. Vamos utilizar essa convenção também para os projetos individuais dos alunos.

```
from util.gerais import imprimir_objetos
from entidades.agência_publicidade import (inserir_agência_publicidade, AgênciaPublicidade, get_agências_publicidade,
                                           selecionar_agências_publicidade)
from entidades.veículo import inserir_veículo, Veículo, get_veículos, selecionar_veículos

def cadastrar_agências_publicidade():
    inserir_agência_publicidade(AgênciaPublicidade(nome='BETC Havas', país_origem='França', ranking_avaliação=1))
    inserir_agência_publicidade(AgênciaPublicidade('Galeria', 'Brasil', 2))
    inserir_agência_publicidade(AgênciaPublicidade('Artplan', 'Brasil', 3))
    inserir_agência_publicidade(AgênciaPublicidade('McCann', 'Estados Unidos', 4))
    inserir_agência_publicidade(AgênciaPublicidade('Africa', 'Estados Unidos', 5))

def cadastrar_veículos():
    inserir_veículo(Veículo(marca_modelo='Hyundai HB20', alimentação='flex', potência=80, importado=False))
    inserir_veículo(Veículo('Chevrolet Onix Plus', 'flex', 116, False))
    inserir_veículo(Veículo('Fiat Strada', 'diesel', 130, False))
    inserir_veículo(Veículo('Fiat Toro', 'diesel', 185, False))
    inserir_veículo(Veículo('BYD Dolphin', 'elétrico', 95, True))
    inserir_veículo(Veículo('Volvo XC40', 'elétrico', 231, True))
    inserir_veículo(Veículo('Volvo FH 540', 'diesel', 540, False))
    inserir_veículo(Veículo('VW Delivery 11180', 'diesel', 175, False))
    inserir_veículo(Veículo('DAF XF 530', 'diesel', 530, False))
    inserir_veículo(Veículo('Volvo FH 460', 'diesel', 460, False))
```

Inicialmente, são definidas as importações de classes e funções, implementadas em outros arquivos, que são utilizadas no arquivo `projeto`. As importações do arquivo `agência_publicidade`, do diretório `entidades`, são delimitadas por parênteses, porque ocupam mais de uma linha.

A seguir, são definidas as funções `cadastrar_agências_publicidade` e `cadastrar_veículos`, para criar e inserir objetos das respectivas classes em seus conjuntos. A função que leva o nome da própria classe (ex: `Veículo`), é conhecida genericamente como construtor, porque é utilizada para construir um objeto em memória. Quando essa função é chamada, é executado o método `__init__` da respectiva classe. Observe que o objeto criado (ex: `veículo`) é passado como parâmetro para um método de inserção (ex: `inserir_veículo`) que appenda o objeto criado no conjunto de objetos (ex: `veículos`).

Para concluir o código do arquivo, é mostrado o corpo principal do projeto, no qual são chamadas as funções de cadastro (definidas anteriormente no próprio arquivo) e as funções de filtragem (ex: `selecionar_veículos`) definidas nos arquivos das entidades.

```
if __name__ == '__main__':
    cadastrar_agências_publicidade()
    cabeçalho = 'Agências de Publicidades : nome - país de origem - ranking de avaliação'
    imprimir_objetos(cabeçalho='\\n' + cabeçalho, objetos=get_agências_publicidade())
    filtros, agência_publicidades_selecionadas = selecionar_agências_publicidade()
    imprimir_objetos(cabeçalho, agência_publicidades_selecionadas, filtros)
    filtros, agência_publicidades_selecionadas = selecionar_agências_publicidade(ranking_máximo_avaliação=3)
    imprimir_objetos(cabeçalho, agência_publicidades_selecionadas, filtros)
    filtros, agência_publicidades_selecionadas = selecionar_agências_publicidade(3, país_origem='Brasil',)
    imprimir_objetos(cabeçalho, agência_publicidades_selecionadas, filtros)
    filtros, agência_publicidades_selecionadas = selecionar_agências_publicidade(3, 'Brasil', prefixo_nome='A')
    imprimir_objetos(cabeçalho, agência_publicidades_selecionadas, filtros)
    cadastrar_veículos()
    cabeçalho = 'Veículos : marca modelo - alimentação - potência - importado'
    imprimir_objetos('\\n' + cabeçalho, get_veículos())
    filtros, veículos_selecionados = selecionar_veículos()
    imprimir_objetos(cabeçalho, veículos_selecionados, filtros)
    filtros, veículos_selecionados = selecionar_veículos(importado=False)
    imprimir_objetos(cabeçalho, veículos_selecionados, filtros)
    filtros, veículos_selecionados = selecionar_veículos(False, alimentação='diesel')
    imprimir_objetos(cabeçalho, veículos_selecionados, filtros)
    filtros, veículos_selecionados = selecionar_veículos(False, 'diesel', potência_máxima=200)
    imprimir_objetos(cabeçalho, veículos_selecionados, filtros)
```

Observe as combinações de filtragem de objetos utilizadas para as classes `AgênciaPublicidade` e `Veículo`. Cada vez que um novo filtro é passado com argumento, esse argumento é atribuído ao nome do parâmetro correspondente na função de filtragem. Como já vimos anteriormente, o nome do parâmetro é uma opção para tornar mais legível a passagem dos argumentos para uma dada função.

Observe que a estratégia escolhida para testar o funcionamento da filtragem de objetos facilita bastante a verificação da utilização bem sucedida da adição de cada filtro como obrigatório. Inicialmente, sem nenhum filtro obrigatório a lista de selecionados é equivalente à lista de cadastrados e a medida que cada filtro adicional é tornado obrigatório, a lista de selecionados diminui pelo menos de um objeto. Obviamente, para obter esse efeito, que facilita o teste da filtragem, você precisará escolher adequadamente os valores dos atributos dos objetos cadastrados e os valores dos filtros obrigatórios.

Para imprimir os conjuntos de objetos cadastrados, ou selecionados nas filtrações, é utilizada a função `imprimir_objetos`, definida no arquivo `gerais`, do diretório `util`. Nesta função é utilizada a função padronizada `enumerate`, que além de retornar o elemento do conjunto (neste caso: objeto), retorna também o índice desse elemento no conjunto. Esse índice (incrementado de 1, pois inicia em zero) será mostrado na impressão dos objetos.

A função `imprimir_objetos` controla o loop de impressão do conjunto de objetos e chama a função `imprimir_objeto` para imprimir cada objeto com o seu respectivo índice.

```
def imprimir_objetos(cabeçalho, objetos, filtros=None):
    if filtros is not None: print(filtros)
    print(cabeçalho)
    for índice, objeto in enumerate(objetos):
        imprimir_objeto(indíce, str(objeto))

def imprimir_objeto(indíce, objeto_str):
    formato = '{ } {}'
    ordem = índice + 1
    separador = '-'
    string_formatado = formato.format(f'{ordem:2d}', separador, objeto_str)
    print(string_formatado)
```

Ao executar o projeto, é gerada a seguinte saída inicialmente para o cadastro e as filtragens dos objetos da classe `AgênciaPublicidade`.

```
Agências de Publicidades : nome - país de origem - ranking de avaliação
1 - | BETC Havas | França          | 1 |
2 - | Galeria   | Brasil           | 2 |
3 - | Artplan   | Brasil           | 3 |
4 - | McCann    | Estados Unidos  | 4 |
5 - | Africa    | Estados Unidos  | 5 |
```

Filtros --

```
Agências de Publicidades : nome - país de origem - ranking de avaliação
1 - | BETC Havas | França          | 1 |
2 - | Galeria   | Brasil           | 2 |
3 - | Artplan   | Brasil           | 3 |
4 - | McCann    | Estados Unidos  | 4 |
5 - | Africa    | Estados Unidos  | 5 |
```

Filtros -- ranking máximo de avaliação: 3

```
Agências de Publicidades : nome - país de origem - ranking de avaliação
1 - | BETC Havas | França          | 1 |
2 - | Galeria   | Brasil           | 2 |
3 - | Artplan   | Brasil           | 3 |
```

Filtros -- ranking máximo de avaliação: 3 - país de origem: Brasil

```
Agências de Publicidades : nome - país de origem - ranking de avaliação
1 - | Galeria   | Brasil           | 2 |
2 - | Artplan   | Brasil           | 3 |
```

Filtros -- ranking máximo de avaliação: 3 - país de origem: Brasil - prefixo do nome: A

```
Agências de Publicidades : nome - país de origem - ranking de avaliação
1 - | Artplan   | Brasil           | 3 |
```


Finalmente, são mostradas as saídas para o cadastro e as filtrações dos objetos da classe `Veículo`. Como o valor para o filtro `importado` foi `True`, aparece o valor `nacional` para esse filtro, ao invés de `não importado`.

```
Veículos : marca modelo - alimentação - potência - importado
1 - | Hyundai HB20      | flex      | 80 cv |
2 - | Chevrolet Onix Plus | flex      | 116 cv |
3 - | Fiat Strada       | diesel    | 130 cv |
4 - | Fiat Toro         | diesel    | 185 cv |
5 - | BYD Dolphin       | elétrico  | 95 cv | importado |
6 - | Volvo XC40        | elétrico  | 231 cv | importado |
7 - | Volvo FH 540      | diesel    | 540 cv |
8 - | VW Delivery 11180 | diesel    | 175 cv |
9 - | DAF XF 530        | diesel    | 530 cv |
10 - | Volvo FH 460     | diesel    | 460 cv |
```

```
Filtros --
Veículos : marca modelo - alimentação - potência - importado
1 - | Hyundai HB20      | flex      | 80 cv |
2 - | Chevrolet Onix Plus | flex      | 116 cv |
3 - | Fiat Strada       | diesel    | 130 cv |
4 - | Fiat Toro         | diesel    | 185 cv |
5 - | BYD Dolphin       | elétrico  | 95 cv | importado |
6 - | Volvo XC40        | elétrico  | 231 cv | importado |
7 - | Volvo FH 540      | diesel    | 540 cv |
8 - | VW Delivery 11180 | diesel    | 175 cv |
9 - | DAF XF 530        | diesel    | 530 cv |
10 - | Volvo FH 460     | diesel    | 460 cv |
```

```
Filtros -- nacional
Veículos : marca modelo - alimentação - potência - importado
1 - | Hyundai HB20      | flex      | 80 cv |
2 - | Chevrolet Onix Plus | flex      | 116 cv |
3 - | Fiat Strada       | diesel    | 130 cv |
4 - | Fiat Toro         | diesel    | 185 cv |
5 - | Volvo FH 540      | diesel    | 540 cv |
6 - | VW Delivery 11180 | diesel    | 175 cv |
7 - | DAF XF 530        | diesel    | 530 cv |
8 - | Volvo FH 460      | diesel    | 460 cv |
```

```
Filtros -- nacional - alimentação: diesel
Veículos : marca modelo - alimentação - potência - importado
1 - | Fiat Strada       | diesel    | 130 cv |
2 - | Fiat Toro         | diesel    | 185 cv |
3 - | Volvo FH 540      | diesel    | 540 cv |
4 - | VW Delivery 11180 | diesel    | 175 cv |
5 - | DAF XF 530        | diesel    | 530 cv |
6 - | Volvo FH 460      | diesel    | 460 cv |
```

```
Filtros -- nacional - alimentação: diesel - potência máxima: 200
Veículos : marca modelo - alimentação - potência - importado
1 - | Fiat Strada       | diesel    | 130 cv |
2 - | Fiat Toro         | diesel    | 185 cv |
3 - | VW Delivery 11180 | diesel    | 175 cv |
```

7 - Encapsulamento

Antes de concluir, precisamos comentar um conceito importante no contexto da orientação a objetos: encapsulamento.

Na linguagem orientada a objetos Java, são utilizadas palavras reservadas como modificadores para restringir o acesso a dados e métodos de uma classe, a partir de um objeto da classe. Se você definir um dado em uma classe, e informar com a palavra reservada `private` que esse dado é privado, não será possível ler ou alterar esse dado diretamente a partir de um objeto da classe. Da mesma forma, se você definir um método como privado, não será possível chamá-lo a partir de um objeto da classe.

Em Python, o encapsulamento de atributos de objetos é diferente. Vamos começar ilustrando a implementação de leitura e escrita de um atributo sem o uso de encapsulamento. Na definição ilustrada para a classe `AgênciaPublicidade`, você pode alterar o atributo `país_origem` diretamente. A primeira agência de publicidade foi cadastrada como `país_origem = França`. Você poderá obter a primeira agência de publicidade cadastrada na lista `agências_publicidade` e alterar o seu atributo `país_origem` da seguinte forma:

```
agência_publicidade = get_agências_publicidade()[0]
agência_publicidade.país_origem = 'Espanha'
print(agência_publicidade.país_origem)
```

A saída será:

```
leitura inicial: França
alteração: Espanha
```

Para implementar o encapsulamento em Python, você deve redefinir o atributo, da classe `AgênciaPublicidade`, para `__país_origem` para `__país_origem`, e utilizar decoradores (`@property` e `@setter`) para implementar métodos de leitura e escrita.

```
@property
def país_origem(self):
    print('atributo país_origem lido via método de leitura')
    return self.__país_origem

@país_origem.setter
def país_origem(self, país_origem):
    self.__país_origem = país_origem
    print('atributo país_origem alterado via método de escrita')
```

A saída será:

```
atributo país_origem lido via método de leitura
leitura inicial: França
atributo país_origem alterado via método de escrita
atributo país_origem lido via método de leitura
alteração: Espanha
```

Obviamente, os métodos de leitura e escrita, não devem ser implementados com comandos de impressão (`print`). Ilustramos dessa forma, somente para você poder constatar, que ao prefixar o atributo com o string `'__'` e implementar seus métodos de leitura e escrita, a obtenção e a alteração do atributo passarão a ser realizadas somente pela chamada implícita desses métodos.

Observe que as alterações, para implementar o encapsulamento de atributos em Python, ocorrem somente na classe que contém os atributos. A implementação da leitura e escrita desses atributos, no arquivo [projeto](#) do diretório [controle](#), não se altera.

Não vamos utilizar encapsulamento nos tutorias dessa disciplina, no entanto, é importante que você saiba como fazê-lo, caso deseje utilizá-lo.

8 - Depuração do Código da Implementação

No seu projeto em Python, você pode depurar o código, imprimindo os valores de variáveis que deseja rastrear para poder detectar, no fluxo de execução, qual o comando que introduziu o erro a ser corrigido, ou em casos mais extremos, os erros cometidos na lógica da implementação.

Suponha que você queira inspecionar, porque o seu projeto não está imprimindo as agências de publicidade cadastradas. Para rastrear o problema, você deve imprimir valores de variáveis ou parâmetros de funções ao longo do fluxo da execução de interesse. Observe o trecho de código de interesse no controle.

```
if __name__ == '__main__':
    cadastrar_agências_publicidade()
    cabeçalho = 'Agências de Publicidades : nome - país de origem - ranking de avaliação'
    imprimir_objetos(cabeçalho='\n' + cabeçalho, objetos=get_agências_publicidade())
```

O primeiro passo, é verificar se a função [imprimir_objetos](#) está recebendo os objetos cadastrados no seu argumento [objetos](#). Como o parâmetro [objetos](#) recebe como argumento um conjunto de objetos, é necessário imprimir cada objeto do conjunto. Observe a inserção dessa verificação na função.

```
def imprimir_objetos(cabeçalho, objetos, filtros=None):
    for objeto in objetos: print(objeto) # impressão dos objetos recebidos
    if filtros is not None: print(filtros)
    print(cabeçalho)
    for índice, objeto in enumerate(objetos):
        imprimir_objeto(indíce, str(objeto))
```

Se o resultado da execução for a impressão correta dos objetos cadastrados, significa que a função [imprimir_objetos](#) não está imprimindo os objetos que recebe, ou que a função [imprimir_objeto](#), que ela chama não está imprimindo o objeto.

```
def imprimir_objeto(indíce, objeto_str):
    formato = '{ } {}'
    ordem = índice + 1
    separador = '-'
    string_formatado = formato.format(f'{ordem:2d}', separador, objeto_str)
    print(string_formatado)
```

No entanto, se o conjunto recebido estiver nulo, o próximo passo é verificar a função [get_agências_publicidade](#).

```
def get_agências_publicidade(): return agências_publicidade
```

Como essa função é muito simples, basta uma inspeção visual para verificar sua correção. O próximo passo é verificar a função `cadastrar_agências_publicidade`, que preenche o conjunto `agências_publicidade` retornado pela função `get_agências_publicidade`. Neste caso, você deve verificar se a chamada do construtor da classe `AgênciaPublicidade` está coerente com a definição do método `__init__` da classe.

```
def cadastrar_agências_publicidade():
    inserir_agência_publicidade(AgênciaPublicidade(nome='BETC Havas', país_origem='França',
    ranking_avaliação=1))
    inserir_agência_publicidade(AgênciaPublicidade('Galeria', 'Brasil', 2))
    inserir_agência_publicidade(AgênciaPublicidade('Artplan', 'Brasil', 3))
    inserir_agência_publicidade(AgênciaPublicidade('McCann', 'Estados Unidos', 4))
    inserir_agência_publicidade(AgênciaPublicidade('Africa', 'Estados Unidos', 5))
```

Caso esteja, o próximo passo é verificar a função `inserir_agência_publicidade`.

```
def inserir_agência_publicidade(agência_publicidade):
    agências_publicidade.append(agência_publicidade)
```

Como essa função é muito simples, basta uma inspeção visual para verificar sua correção. O próximo passo, será verificar se a agência de publicidade recebida está correta, inserindo uma verificação na função.

```
def inserir_agência_publicidade(agência_publicidade):
    print(agência_publicidade) # impressão do objeto recebido
    agências_publicidade.append(agência_publicidade)
```

Caso a agência de publicidade recebida não esteja correta, o último passo é verificar se o método `__init__` da classe `AgênciaPublicidade` está inicializando corretamente os parâmetros que recebe.

```
def __init__(self, nome, país_origem, ranking_avaliação):
    self.nome = nome
    self.país_origem = país_origem
    self.ranking_avaliação = ranking_avaliação
```

A depuração de código é essencial no aprendizado do aluno de programação. Exercite para ser tornar proficiente na correção dos seus projetos.